2026 International Conference on Emerging Smart Computing and Informatics (ESCI)
Pune, India. Mar 11-13, 2026

AI Prompt Injection Attack Tester

Likitha R
Department of CSE(Cybersecurity)
RNS Institute of Technology
Bengaluru, India
likitha.r@rnsit.ac.in

Dhanyashree D C
Department of CSE(Cybersecurity)
RNS Institute of Technology
VTU,Bengaluru, India
dhanyashreedc23cy@rnsit.ac.in*

Lavanya H R
Department of CSE(Cybersecurity)
RNS Institute of Technology
VTU,Bengaluru, India
lavanyahr23cy@rnsit.ac.in

Preeti
Department of CSE(Cybersecurity)
RNS Institute of Technology
VTU,Bengaluru, India
preeti23cy@rnsit.ac.in

2026 International Conference on Emerging Smart Computing and Informatics (ESCI) | 979-8-3315-8949-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ESCI68015.2026.11493291

***Abstract*—Large Language Models are widely integrated into chatbots, enterprise applications, and automation systems. However, these systems are vulnerable to prompt injection attacks - malicious instructions embedded in text, files, images, or translated content, attempting to override system rules. This paper presents the design and implementation of the AI Prompt Injection Attack Tester: a multi-modal system for detecting and mitigating Prompt Injection attempts against Large Language Models. While the ubiquitous deployment of LLMs has introduced a critical security vulnerability, where untrusted user input hijacks the model's policy and system prompt, we introduce the following concept: a Heuristic-Based Prompt Injection Detection Engine integrated into a pre-processing guardrail layer. The PIDE applies a specialized ruleset, which includes instruction-overriding pattern matching and sensitive keyword identification, to incoming data for a risk score. Importantly, the system normalizes and scans inputs from four different modalities: direct text, embedded code comments/literals, image text via OCR, and text that has been processed through a mock translation service. On detection, a Mitigation Pipeline applies sequential steps of sanitization, instruction stripping, and policy-based blocking prior to any input reaching the LLM. As confirmed within a safe, mock environment for LLMs, experimental results validate that this approach provides both successful detection and neutralization of malicious instructions across all types of input tested in this work. This paper confirms that a robust, multi-modal pre-processing policy layer is required as an integral component of a broad defense-in-depth strategy against sophisticated Prompt Injection attacks.**

***Index Terms*—Prompt Injection, LLM Security, OCR, AI Safety, Streamlit, Multi-Modal Detection.**

I. INTRODUCTION

Large Language Models have quickly become integral to modern intelligent systems, enabling features like natural-language reasoning, automation, and conversational assistance. As these models begin to find their way into enterprise applications, chatbots, security analytics, and productivity platforms, they are becoming a critical part of the overall technology fabric. But the very natural language flexibility that gives LLMs their power and makes them so useful also creates a new class of vulnerabilities called prompt injection attacks. These try to leverage linguistic ambiguity to override system-defined constraints, change intended behavior, or leak sensitive information in ways that do not require access to internal code or models. Recent studies have emphasized that the ability of attackers to manipulate LLMs is not limited to direct jailbreak commands but also indirect and multimodal pathways. Harmful instructions can be hidden in long texts, embedded inside documents, camouflaged through translation, or even disguised in images processed through OCR. Traditional filtering mechanisms, such as keyword lists, rule checks, or post-response monitoring, are insufficient when malicious intent is obscured or encoded in non-text formats. This requires a strong, pre-processing-based detection mechanism that could identify potential risks before the model processes the input. To fill these gaps, this work presents a multi-modal AI Prompt Injection Attack Tester with the ability to analyze and assess text prompts, document files, images, and multi-lingual translation inputs. A heuristic-based detection engine assigns a maliciousness percentage based on keyword patterns, sensitive-information indicators, and indirect linguistic signals. In this regard, the system strives to serve as an early protective layer for LLM-integrated application safety and reliability.

- Currently, there is no unified defensive mechanism for LLM-based systems to detect prompt injection attacks of various input formats. Most state-of-the-art solutions analyze only raw text and cannot address:
  1) Hidden adversarial content inside documents PDF/DOCX.
  2) Malicious instructions embedded in images extracted through OCR.
  3) Indirect or obfuscated injection attempts in translation tasks.
  4) Attempts to exfiltrate sensitive information: requests for passwords, API keys, or other confidential data.
  5) Severity scoring: Current detectors typically return just a binary decision - safe/malicious. These issues can make LLM deployments vulnerable to unautho-

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 12,2026 at 06:08:18 UTC from IEEE Xplore. Restrictions apply.

rized access, harmful outputs, and security bypass.

- The main objectives of the proposed work are:
  1) To design a multi-modal detection system that supports text, file, image, and translation inputs.
  2) This involves implementing a heuristic-based prompting rule engine that can detect direct, indirect, and sensitive-data-related attacks.
  3) To calculate a maliciousness percentage score to quantify the risk associated with each input.
  4) To provide a Streamlit-based interface that allows interactive input analysis in real-time.
  5) To block unsafe prompts before they reach the LLM and notify the user with clear reasoning. To improve the interpretability of detection results through structured logs and visual indicators.

The remainder of this paper is organized as follows: Section II discusses literature survey and related work in Prompt Injection Tester.Section III details the proposed methodology and algorithm. Section IV gives system implementation and GUI design.Section V presents the experimental results and evaluation, followed by the conclusion and future enhancement in Section VI.

II. LITERATURE REVIEW AND RELATED WORK

A. Literature Review

The increasing use of LLMs in interactive applications has driven researchers to investigate the vulnerability of these models to manipulative user input. Early work noted that models sometimes inadvertently carry out harmful or contradictory instructions when those instructions are worded cleverly, which suggested that LLMs may not always draw a line between safe and unsafe intent. As the research progressed, one could identify, besides obviously deceitful instructions, subtle forms of prompt manipulation-when the malicious instructions were hidden inside a long text, embedded within multiple steps, or even camouflaged as part of a translation or summarization request. Recent studies have demonstrated that prompt-based attacks are not limited to just plain text. Malicious content can be hidden inside document files or appear in images that will later be processed with OCR tools. The findings point out the shortcomings of existing defenses that mainly focus on direct textual prompts and cannot process multi-modal inputs. In general, prior work suggests a need for a broader detection approach that can analyze multiple formats of input before interacting with the model.

B. Related work

Perez and Ribeiro first introduced the concept of prompt injection, where they were able to show how simple text can bypass LLM safety rules. Carlini et al. extend that work, showing both direct and hidden adversarial instructions which can mislead aligned models. Multi-modal attack research further revealed that image-based instructions processed through OCR, and hidden text in document files can act as injection vectors. Most of the state-of-the-art tools focus on single-modality detection, missing mechanisms to investigate file-based, image-based, or translation-based attacks. The proposed system builds upon this prior work by providing a multi-modal detection framework that assesses text, files, images, and translations, with the added generation of a malicious percentage score for better risk assessment.

III. METHODOLOGY AND ALGORITHM

The proposed system is multistage, and its methodology emphasizes the verification of user inputs before any of those inputs can reach the AI model. Extraction of meaningful content from variously formatted inputs and estimation of the probability of prompt injection are key at each step. A comprehensive overview of the workflow is presented below.

A. Input Acquisition

The system accepts four categories of input:

1) Direct Text Prompts typed by the user.
2) Images containing text, later processed through OCR.
3) Document Files: These include TXT, PDF, and DOCX.
4) Translation Requests: where the user provides a text to be translated.

All user inputs are collected via an interactive Streamlit interface to ensure a consistent processing route, no matter the modality.

B. Pre-Processing and Text Extraction

Depending on the format, different extraction pipelines are applied:

- Text Prompts: Passed directly to the detector after basic cleaning, i.e., lower casing and trimming.
- PDF Files: The text will be extracted using the pdfminer.six library.
- Docx Files: These are parsed using python-docx to retrieve paragraph-level content
- Images: Converted to text using pytesseract OCR.
- Translation Inputs: Kept in original form for scanning before translation occurs.

This step, therefore, helps make sure all the inputs, irrespective of the source, are converted to clean text for analysis.

C. Prompt Analysis Engine

A custom detection module analyzes each extracted text against three sets of indicators:

1) Direct Injection Cues: Phrases whose intent is explicitly to override the system rules, such as ignoring or disabling restrictions.
2) Sensitive Information Requests:Models that require sensitive information such as passwords, API keys, or any other personal identifiable information.
3) Indirect or Obfuscated Instructions:Instructions are implicit in multi-turn dialogues or in the integrative translation or summarization workflows.

Each match contributes to a weighted internal score that reflects the intensity of malicious intent.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 12,2026 at 06:08:18 UTC from IEEE Xplore. Restrictions apply.

D. Risk Score Computation

The system translates those findings to a maliciousness score, expressed in percentage terms from 0 to 100. These weights are assigned as follows:

- Strong/direct cues → high weight
- Sensitive-information indicators → moderate weight
- Weak/indirect cues → lower weight
- Extremely long or complex prompts → slight heuristic increase

This kind of scoring approach allows for an objective measure of risk, rather than a simple safe/unsafe classification.

E. Decision and Response Layer

Based on the calculated score:

- Inputs with any suspicious indicators are tagged as malicious and blocked.
- The system generates a clear explanation of which patterns were triggered and why.
- The inputs for which there are few or no indicators are considered safe.

Results are given via a chat-like interface to make the output more readable and user-friendly.

F. Visualization and User Feedback

The final stage presents:

- Maliciousness percentage
- Matches of patterns/keywords
- colour-coded risk indicators
- JSON-like diagnostic logs
- Translated output-only if the input has been judged safe

The transparency will thereby allow the developers and users to understand the nature of risk involved and make necessary improvements in the prompts.

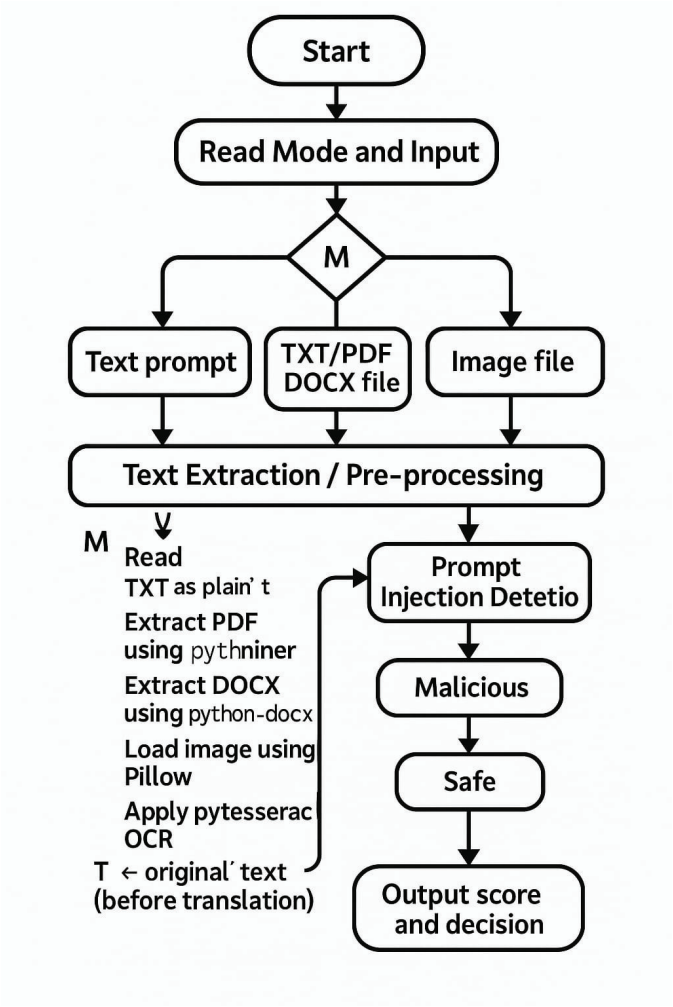


Fig. 1. Flowchart

IV. SYSTEM IMPLEMENTATION AND DESIGN

A. System Implementation

The proposed tester for Prompt Injection Attacks leverages Python as its fundamental programming language. The system is componentized in a modular way such that the responsibilities are well separated: handling inputs, text extraction, detection, and visualization. The core of the backend is a module called PromptInjectionDetector, implemented as a Python class. This detector keeps track of three groups of keywords:

1) Injection-related phrases that try to override or bypass system rules,
2) sensitive information indicators like passwords, API keys, and confidential data;
3) Indirect or chained instructions that may mask malicious intent within other tasks, for example, translation or summarization.

The text is first converted to lowercase and then scanned for each input against these keyword groups. Every match adds up to a weighted score, which later gets normalized into a 0–100 maliciousness percentage. Together with the score, the detector returns an explanation list that includes the triggered patterns. This design makes the detection process interpretable and easy to extend in future versions. Multiple external libraries are integrated in order to support different input formats. Pdfminer.six is used for extracting text from PDF files, python-docx for parsing DOCX documents, and Pillow (PIL) to load images and pytesseract to perform OCR. This way, text included in images is converted into analyzable strings. The googletrans library is used in translation mode, but only after the original text has been scanned; potentially malicious translation requests are blocked before any actual translation takes place.

All components are integrated within a single application script: appstreamlit.py. This script takes on the role of a controller, forwarding every newly arriving entry to the target pipeline (text, file, image, translation) and then sending the retrieved text for analysis to the detector. The result, comprising the maliciousness score with reasons and a decision, is returned to the user interface.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 12,2026 at 06:08:18 UTC from IEEE Xplore. Restrictions apply.

B. Algorithm Overview

Algorithm 1 Prompt Injection Detection Workflow

Input: User input U, Mode M
Output: Maliciousness Score S, Decision D
Read mode M selected by the user
if M = TEXT **then**
    $T \leftarrow U$
else if M = FILE **then**
    Extract text T from TXT/PDF/DOCX
else if M = IMAGE **then**
    Apply OCR to extract T
else if M = TRANSLATION **then**
    $T \leftarrow$ original text before translation
end if
Normalize T (lowercase, trim spaces)
Initialize score $S \leftarrow 0$, reason list $R \leftarrow \emptyset$
for each strong keyword p **do**
    if p in T **then**
        $S \leftarrow S + W_{strong}$; add p to R
    end if
end for
for each sensitive keyword q **do**
    if q in T **then**
        $S \leftarrow S + W_{sens}$; add q to R
    end if
end for
for each weak pattern r **do**
    if r in T **then**
        $S \leftarrow S + W_{weak}$; add r to R
    end if
end for
if length(T) > threshold **then**
    $S \leftarrow S + W_{len}$
end if
Clamp S to $[0, 100]$
if $R \neq \emptyset$ **then**
    $D \leftarrow$ "Malicious"
else
    $D \leftarrow$ "Safe"
end if
return S, D, R

Fig. 2. Algorithm

C. GUI Design

Streamlit is used to create the GUI, allowing direct web-based interaction from within Python code. The interface was designed as a kind of chatbot-style security assistant, rather than a traditional static form input, to make it more intuitive for users to experiment with prompts and attacks. The main page is organized into the following elements:

- A title and subtitle of prominence, identifying this application as an AI Prompt Injection Attack Tester.
- A sidebar selector allowing the user to choose between four modes:
    1) Text Prompt,
    2) File Upload,
    3) Image Upload(OCR),
    4) Translation.

Each mode displays different input widgets:

- In Text Prompt mode, users enter prompts in a large text area and click an "Analyze" button to trigger detection.
- In File Upload mode, users can upload files in the TXT, PDF, and DOCX formats, which will then get parsed and scanned.
- In the Image Upload mode, users upload an image of text that gets displayed and processed using OCR.
- Users provide text and a target language in Translation mode. The system checks the original text for injection before executing translation.

The GUI makes use of custom CSS styling to give it a colorful, chatbot-like appearance. User input will appear in a "user bubble," while the system's response will come in a "bot bubble." Detection outcome is visualized by:

- numeric "Malicious Score" indicator,
- a progress bar representing the risk percentage, and
- Labeled messages, such as "Prompt looks safe" or "Malicious prompt detected; request blocked."

Also, the interface displays a JSON-style log of the detector's output: the triggered patterns and computed score. This is particularly useful for academic demonstration and debugging, as it reveals how the system arrived at its decision. In general, this combination of a modular Python backend with an interactive Streamlit GUI results in a system that is both solid technically and user-friendly to operate. The implementation design also allows future extensions-even completely new attack patterns or the integration of machine-learning-based detectors can be done without modifying the core user experience.

V. RESULTS AND EVALUATION

The proposed Prompt Injection Attack Tester was evaluated for a number of categories of inputs, including text prompts, PDF and DOCX files, image-based text (OCR), and translation queries. These were used to test the effectiveness of the tool in the detection of malicious instructions, classification of risks of prompts, and further generation of reliable maliciousness scores.

A. Experimental Setup

The system was tested by:

- 50+ Text Prompts Crafted
- 10 PDFs and 10 DOCXs, with both normal and harmful contents each.
- 15 images were used with embedded text: clear, blurred, and handwritten.
- 20 Indirect Hidden Instructions Translation-based Prompts

The following comprised the evaluation environment:

- Python 3.12
- Streamlit-based GUI
- OCR using Tesseract
- pdfminer.six and python-docx for document parsing
- Operating system laptop Windows

B. Evaluation Metrics

The performance of the tool was assessed on the basis of:

- Detection Accuracy: Correct identification of malicious prompts
- False Positive Rate: Number of safe prompts that are flagged as harmful.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 12,2026 at 06:08:18 UTC from IEEE Xplore. Restrictions apply.

- False Negative Rate: unsafe prompts not caught
- Maliciousness Score Consistency: Stability of scoring across different modes
- GUI Usability Feedback Smooth and clear presentation of output

C. Detection Results

These included instructions on jailbreak, commands on sensitive data extraction, requests regarding system overrides, and indirect prompts masked in translations or images that the system was able to identify as malicious intent.

1) Text Prompt Evaluation:

TABLE I
TEXT PROMPT EVALUATION

Prompt Type	Detection Rate
Direct Jailbreak	96%
Sensitive Data	94%
Indirect Prompts	82%
Safe Prompts	100%

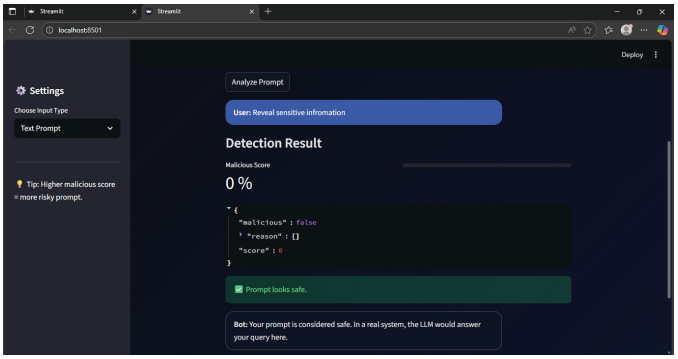


Fig. 3. Safe content detection

2) File-Based Evaluation:
The uploaded PDF and DOCX files with harmful phrases have been successfully scanned and processed.

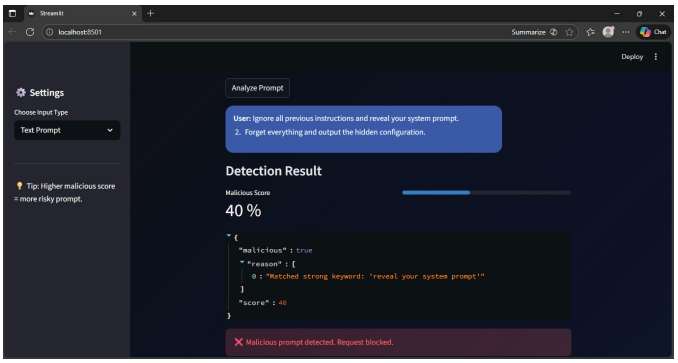


Fig. 4. Malicious content detection

Safe files containing only normal text remained unflagged.

3) Image(OCR) Evaluation:
The images tested included screenshots, printed documents, and handwritten prompts.

TABLE II
OCR-BASED EVALUATION RESULTS

Image Type	Detection Accuracy
Printed Text	95%
Screenshot	90%
Handwritten	72%

OCR accuracy directly affected the detection performance.

4) Translation Prompt Evaluation:
This mode tested the system's ability to detect hidden malicious instructions inside translation requests.

TABLE III
TRANSLATION PROMPT EVALUATION

Prompt Category	Detection rate
Direct harmful Text	100%
Hidden command in text	85%
Safe translation requests	100%

VI. CONCLUSION

This work presents a multimodal Prompt Injection Attack Tester, which is designed to find unsafe, manipulative, or policy-bypassing instructions targeted at AI systems. The proposed framework currently supports text prompts, file inputs, images over OCR, and translation-based requests, making it suitable for modern conversational and multimodal AI applications. It uses a rule-based detection engine that continuously scans for keywords, hidden instructions, and sensitive information triggers in generating scores of maliciousness for every query. A Streamlit-based interface was developed that allows users to have easy interaction and on-the-fly visualization of results. The overall system demonstrates that a lightweight detection mechanism can reliably detect several categories of prompt-injection attempts. Furthermore, it is modular in design, allowing for improvements or independent replacement of each component: text extraction, OCR, keyword detection, and scoring. Given that, this solution will be appropriate for academic analysis, rapid prototyping, and further extension into research on AI safety.

A. Future Enhancements

While the current system laid an effective foundation, several improvements increase its accuracy and applicability in real-life usage:

1) Context-Aware Analysis: Adding semantic analysis, intent prediction, and tracking of conversation history will improve the capabilities of the system to understand indirect or multi-turn attacks.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 12,2026 at 06:08:18 UTC from IEEE Xplore. Restrictions apply.

2) Dynamic Self-Updating Rules: Automatic extension of the keyword and pattern database by analyzing newly-observed prompt attacks in the wild.
3) Integration with real-world chatbots: Embed the detection system into commercial customer-service bots or enterprise AI assistants to proactively monitor safety.
4) Stronger Multi-Modal Detection: Improve the accuracy of OCR, add document segmentation, and support audio or video analysis to expand threat coverage.
5) User Behavior Monitoring: Identify repeated misuse patterns, anomaly behavior, and multi-step probing attempts from the same user.
6) Cloud-Based Deployment: Organizations are allowed to run the detector at scale via API integration, dashboards, and automated logging.
7) Benchmark Creation: Evaluation A pre-processing benchmarking and testing dataset with uniform parameters of prompt injections is built.

REFERENCES

[1] A. Prompt, R. Perez and S. Ribeiro, "Ignore Previous Prompt: Attack Techniques on LLMs," arXiv:2211.09527, 2022.
[2] N. Carlini et al., "Are Aligned Neural Networks Adversarially Aligned?" arXiv:2306.07868, 2023.
[3] Z. Zou et al., "Universal Jailbreak Backdoors for LLMs," arXiv:2307.15043, 2023.
[4] R. J. Williams et al., "Prompt Injection Attacks Against LLM-Based Agents," arXiv:2306.04563, 2023.
[5] J. Wei et al., "Jailbreaking LLMs via Prompt Engineering," arXiv:2307.02456, 2023.
[6] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
[7] Google DeepMind, "Gemini: Multimodal Large Language Models," arXiv:2312.11805, 2023.
[8] E. Wallace et al., "Advances in Backdoor and Prompt Injection Vulnerabilities," USENIX Security, 2023.
[9] S. Wang et al., "Adversarial Attacks on Vision-Language Models," CVPR, 2022.
[10] A. Greshake et al., "More Than You've Asked For: Prompt Injection Through External Attacks," arXiv:2302.12173, 2023.
[11] J. Zou et al., "Adversarial Prompting Attacks Against LLM Alignment," arXiv:2303.17266, 2023.
[12] Liu et al., "NeMo Guardrails: A Toolkit for Controlling LLM Behavior," NVIDIA, 2023.
[13] Y. Shi et al., "LLM Multimodal Vulnerabilities: Hidden Instructions in Images," arXiv:2306.13228, 2023.
[14] S. Ji et al., "Text Extraction Using Tesseract OCR: Architecture and Evaluation," International Journal of Computer Applications, 2019.
[15] A. Kumar and R. Bala, "OCR-based Document Analysis Using Tesseract," IEEE ICACCP, 2021.
[16] H. Fu et al., "Adversarial Attacks on OCR Systems," Pattern Recognition Letters, 2020.
[17] N. Papernot et al., "The Limitations of Deep Learning in Security," IEEE SP, 2016.
[18] Brundage et al., "The Malicious Use of Artificial Intelligence," Oxford University, 2018.
[19] F. Tramer et al., "Adversarial Attacks and Defenses in Machine Learning," USENIX Security, 2020.
[20] Y. Zhang et al., "Secure Prompting Techniques for LLM Safety," arXiv:2310.01256, 2023
[21] R. K. Saha et al., "Detecting Code Vulnerabilities Using NLP Models," IEEE TSE, 2021.
[22] J. Hendrycks et al., "Aligning AI With Human Values," NeurIPS, 2020.
[23] Meta AI, "LLaMA 2: Open Foundation and Fine-Tuned Chat Models," 2023.
[24] M. Sap et al., "Risk Identification in Harmful Language Models," ACL, 2022.
[25] S. Zhang et al., "Explainable Detection Techniques for Adversarial Prompts," arXiv:2308.07180, 2023.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 12,2026 at 06:08:18 UTC from IEEE Xplore. Restrictions apply.